

นายสิรภัทร ลีล้าน 673380067-2 sec.2

ส่วนที่ 1: หลักการออกแบบ

อธิบายว่าใช้ SOLID ส่วนไหนบ้างและเกี่ยวข้องกับ Entity อย่างไร

S — Single Responsibility Principle

แยก ProductDetail ออกจาก Product เพราะเป็นข้อมูลเสริม (description, warranty, weight, dimensions) ที่ไม่จำเป็นต้องใช้ทุกครั้ง query สินค้า การแยกออกทำให้แต่ละ class มีหน้าที่ชัดเจน: Product ดูแลข้อมูลหลักของสินค้า ส่วน ProductDetail ดูแลข้อมูลเสริมเท่านั้น เช่นเดียวกัน ProductService มีหน้าที่แค่ business logic (การตั้งค่า relationship ก่อน save) ส่วน ProductController มีหน้าที่รับ-ส่ง HTTP request/response เท่านั้น ไม่ปนกัน

O — Open/Closed Principle

ระบบส่วนลดออกแบบผ่าน DiscountStrategy interface ทำให้เพิ่มส่วนลดแบบใหม่ได้โดยสร้าง class ใหม่ implement interface นี้ (เช่นจะเพิ่ม HolidayDiscountStrategy) โดยไม่ต้องแก้โค้ดเดิมใน Product หรือ DiscountContext เลย เช่นเดียวกับความสัมพันธ์ 1:N — ถ้าจะเพิ่มตาราง Review เข้ามาใหม่ ก็เพิ่ม entity ใหม่และผูก @ManyToOne กลับมาที่ Product ได้โดยไม่ต้องแก้โครงสร้างเดิมของ Product

L — Liskov Substitution Principle

ProductRepository, ProductDetailRepository, ReviewRepository ทุกตัว extends JpaRepository<T, Long> และถูกใช้งานผ่าน interface เดียวกันในทุก service — ไม่ว่าจะเป็น implementation ที่ Spring Data JPA generate ให้อัตโนมัติ ตัวไหน ก็ใช้แทนกันได้ตาม contract ของ JpaRepository

I — Interface Segregation Principle

แยก repository ออกเป็น 3 interface ตามตาราง (ProductRepository, ProductDetailRepository, ReviewRepository) แทนที่จะรวมทุก method การเข้าถึงข้อมูลไว้ใน interface เดียว ทำให้แต่ละ repository มีขอบเขตความรับผิดชอบที่ชัดเจนและเล็กพอดีกับการใช้งาน

D — Dependency Inversion Principle

ProductService ไม่ได้ new ProductRepositoryImpl() ขึ้นมาใช้เองตรงๆ แต่ขึ้นกับ ProductRepository (interface) ผ่าน Constructor Injection โดย Spring จะ inject implementation ที่ generate จาก JPA ให้อัตโนมัติตอน runtime ทำให้ service ไม่ผูกติดกับ implementation จริง เปลี่ยน implementation ได้โดยไม่กระทบ business logic

อธิบายความแตกต่างของ 1:1 กับ 1:N — ใช้กรณีไหนควรใช้แบบไหน

One-to-One (Product ↔ ProductDetail): สินค้า 1 รายการมีข้อมูลเสริมได้เพียง 1 ชุดเท่านั้น ใช้เมื่อข้อมูลเป็น "ส่วนขยาย" ของ entity หลักที่ไม่ต้องใช้ทุก query — ในระบบนี้ Product เป็นฝั่งเจ้าของความสัมพันธ์ (เก็บ detail_id เป็น foreign key) ส่วน ProductDetail เป็นฝั่ง inverse (mappedBy = "detail")

One-to-Many (Product → Review): สินค้า 1 รายการมีรีวิวได้หลายรายการไม่จำกัด ใช้เมื่อข้อมูล child เพิ่มได้เรื่อยๆ ตามเวลา — ในระบบนี้ Review เป็นฝั่งเจ้าของความสัมพันธ์เสมอ (เก็บ product_id เป็น foreign key ผ่าน @ManyToOne) ส่วน Product เป็นฝั่ง inverse ที่เก็บ List<Review> (mappedBy = "product")

หลักการเลือกใช้: ถ้าความสัมพันธ์มีจำนวนคงที่แค่ 1 ต่อ 1 เสมอ (เช่น รายละเอียดเสริม) ใช้ 1:1 แต่ถ้าฝั่งใดฝั่งหนึ่งสามารถมีได้หลายรายการเพิ่มขึ้นเรื่อยๆ (เช่น รีวิวจากลูกค้าหลายคน) ต้องใช้ 1:N โดยฝั่ง "many" จะเป็นฝั่งที่เก็บ foreign key เสมอ

อธิบาย Strategy Pattern สำหรับคำนวณส่วนลด

ระบบมี DiscountStrategy interface กำหนด method calculateDiscount(double price) และมี concrete implementation 3 ตัว: NoDiscountStrategy (ไม่ลด), StudentDiscountStrategy (ลด 10%), SeasonalSaleStrategy (ลด 20%) การเลือก strategy ทำผ่าน DiscountContext ซึ่งรับ strategy เข้ามาผ่าน setStrategy() แล้วเรียก executeStrategy(price) เพื่อคำนวณราคาสุดท้าย

ใน `Product.getDiscountedPrice()` จะอ่านค่า `discountType` (String ที่มาจาก dropdown ในฟอร์ม) แล้วเลือกสร้าง strategy object ที่ตรงกันด้วย if-else จากนั้นส่งให้ `DiscountContext` คำนวณ วิธีนี้ทำให้ตรรกะการคำนวณส่วนลดแต่ละแบบแยกออกจากกันอย่างชัดเจน (แต่ละ class คำนวณสูตรของตัวเอง) และเพิ่มส่วนลดแบบใหม่ในอนาคตได้โดยไม่แตะโค้ดเดิม

อธิบาย Execution Flow ตั้งแต่ HTTP Request → DB

ตัวอย่าง flow ของการเพิ่มสินค้าใหม่ (Create):

ผู้ใช้กรอกฟอร์มที่หน้า `add.html` แล้วกด submit → ส่ง POST `/products/save`

`ProductController.save()` รับข้อมูลผ่าน `@ModelAttribute Product product` (Spring bind ข้อมูลจากฟอร์ม รวมถึง `detail.*` และ `reviews[0].*` เป็น nested object ให้อัตโนมัติผ่าน Thymeleaf `th:field`)

Controller เรียก `productService.save(product)`

`ProductService.save()` วนลูป `product.getReviews()` เพื่อ set ค่า `review.setProduct(product)` ให้แต่ละ review ก่อน (จำเป็นเพราะ Review เป็นฝั่งเจ้าของ FK — ถ้าไม่ set ตรงนี้ `product_id` จะเป็น NULL)

เรียก `productRepository.save(product)` — เพราะตั้ง `cascade = CascadeType.ALL` ไว้ทั้งที่ `detail` และ `reviews` การ save Product ครั้งเดียวจะ insert ทั้ง 3 ตารางให้อัตโนมัติ (Hibernate จะสร้าง SQL INSERT ให้ `products`, `product_details`, `reviews` ตามลำดับ)

Controller return `"redirect:/products"` → browser ถูก redirect กลับไปหน้า list ซึ่งจะ query ข้อมูลใหม่ทั้งหมดมาแสดง

ส่วนที่ 2: Code + คำอธิบาย

Entity: `ProductDetail.java`

```

Java
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;
    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    @OneToOne(mappedBy = "detail")
    private Product product;
    // getters/setters ...
}

```

คำอธิบาย

- `@Entity + @Table(name = "product_details")` — ระบุว่า class นี้ map กับตาราง `product_details` ใน database
- `@Id + @GeneratedValue(strategy = GenerationType.IDENTITY)` — กำหนด primary key ที่ auto-increment ให้ database เป็นผู้สร้างค่าให้ (ตรงกับ `SERIAL/IDENTITY` ของ PostgreSQL)
- `@OneToOne(mappedBy = "detail")` — ระบุว่า `ProductDetail` เป็นฝั่ง **inverse** ของความสัมพันธ์ 1:1 ค่า "detail" หมายถึงชื่อ field `detail` ใน class `Product` ที่เป็นเจ้าของความสัมพันธ์จริง ฝั่งนี้จะไม่ค้อลมันน์ foreign key ของตัวเอง

Entity: Review.java

```

Java
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```

private String reviewer;
private Integer rating;
private String comment;
private LocalDate reviewDate;

@ManyToOne
@JoinColumn(name = "product_id")
private Product product;
// getters/setters ...
}

```

คำอธิบาย

- @ManyToOne — ระบุว่า Review หลายรายการเชื่อมกับ Product เพียง 1 รายการ ฟังก์ชันนี้เป็นฝั่ง **เจ้าของความสัมพันธ์** (owning side) เสมอในความสัมพันธ์ 1:N
- @JoinColumn(name = "product_id") — กำหนดชื่อคอลัมน์ foreign key ที่จะถูกสร้างในตาราง reviews ให้ชี้กลับไปยัง [products.id](#)

Entity: Product.java

```

Java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private String brand;
    private Integer stock;
    private Double price;
    private String discountType;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Review> reviews = new ArrayList<>();

    // getters/setters ...
}

```

```

public Double getDiscountedPrice() {
    DiscountStrategy strategy;
    if (discountType == null) {
        strategy = new NoDiscountStrategy();
    } else if (discountType.equals("MEMBER")) {
        strategy = new StudentDiscountStrategy();
    } else if (discountType.equals("SEASONAL")) {
        strategy = new SeasonalSaleStrategy();
    } else {
        strategy = new NoDiscountStrategy();
    }
    DiscountContext context = new DiscountContext();
    context.setStrategy(strategy);
    return context.executeStrategy(price);
}
}

```

คำอธิบาย

- @OneToOne(cascade = CascadeType.ALL) + @JoinColumn(name = "detail_id", referencedColumnName = "id") — Product เป็นฝั่งเจ้าของความสัมพันธ์ 1:1 เก็บคอลัมน์ detail_id เป็น FK ซึ่งไปที่ product_details.id ส่วน cascade = CascadeType.ALL หมายถึง เมื่อ save/update/delete Product การเปลี่ยนแปลงจะกระทบไปถึง ProductDetail ที่ผูกกันอัตโนมัติ (save พร้อมกัน ลบพร้อมกัน)
- @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true) — Product เป็นฝั่ง inverse ของ 1:N เก็บ List<Review> โดยไม่มีคอลัมน์ FK ของตัวเอง orphanRemoval = true หมายถึง ถาลบ review ออกจาก list แล้ว save จะลบ record นั้นออกจากฐานข้อมูลด้วย (ไม่ใช่แค่ตัด reference)
- getDiscountedPrice() — ไม่ใช่ field ที่ map กับ database แต่เป็น method คำนวณจาก field price และ discountType ที่มีอยู่แล้ว ใช้ Strategy Pattern เลือก algorithm การคำนวณตามประเภทส่วนลด แล้ว Thymeleaf ใน list.html เรียกใช้ผ่าน \${product.discountedPrice} ได้โดยตรง (ตาม Java Bean convention ที่ getter getX() เข้าถึงได้แบบ property x)

Service: ProductService.java

```

Java
@Service
public class ProductService {

    private final ProductRepository productRepository;

    @Autowired

```

```

public ProductService(ProductRepository productRepository) {
    this.productRepository = productRepository;
}
// ...
}

```

คำอธิบาย: ใช้ **Constructor Injection** แทนการใช้ `@Autowired` บน field ใดๆ เพราะทำให้ `productRepository` เป็น final (immutable หลัง construct) ภาระดีได้ว่า object นี้จะไม่เป็น null หลังสร้าง class เสร็จ (fail-fast ถ้า dependency หาย) และทำให้ unit test เขียนง่ายขึ้น เพราะสามารถส่ง mock `ProductRepository` เข้ามาทาง constructor ได้โดยตรงโดยไม่ต้องพึ่ง Spring container

เมธอด `save()` มีจุดสำคัญคือการวน loop ตั้งค่า `review.setProduct(product)` ให้ทุก review ก่อน save เพราะ Review เป็นฝั่งเจ้าของ FK (`product_id`) — ถ้าไม่ set ค่านี้ Hibernate จะไม่รู้ว่า review แต่ละตัวเป็นของ product ไหน ทำให้ `product_id` บันทึกเป็น NULL

Controller: ProductController.java

```

Java
@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    @Autowired
    public ProductController(ProductService productService) {
        this.productService = productService;
    }
    // @GetMapping, @PostMapping ตาม URL mapping...
}

```

คำอธิบาย: `ProductController` ทำหน้าที่เป็นตัวกลางรับ HTTP request (`@GetMapping`, `@PostMapping`) แล้วส่งต่อให้ `ProductService` ทำ business logic เท่านั้น ไม่มีการเรียก `productRepository` ใดๆ จาก controller เลย — เป็นการรักษา **Layered Architecture** (Controller → Service → Repository) ให้แต่ละชั้นทำหน้าที่ของตัวเองเท่านั้น สอดคล้องกับ SRP และ DIP ที่อธิบายไว้ในส่วนที่ 1

หน้า add.html

localhost:8080/products/add

✚ เพิ่มสินค้าใหม่

ข้อมูลสินค้า (Product)

ชื่อสินค้า

หมวดหมู่

ยี่ห้อ

จำนวนในสต็อก

ราคาปกติ (บาท)

ประเภทส่วนลด

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

ระยะเวลาประกัน (ปี)

น้ำหนัก (กรัม)

ขนาด (กว้าง x ยาว x สูง)

ประเทศต้นกำเนิด

★ รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เมื่อมีรีวิวอย่างน้อย 5 รีวิวสินค้าจะแสดงเป็นกราฟิกดาว (5 ดาว)

ชื่อผู้รีวิว

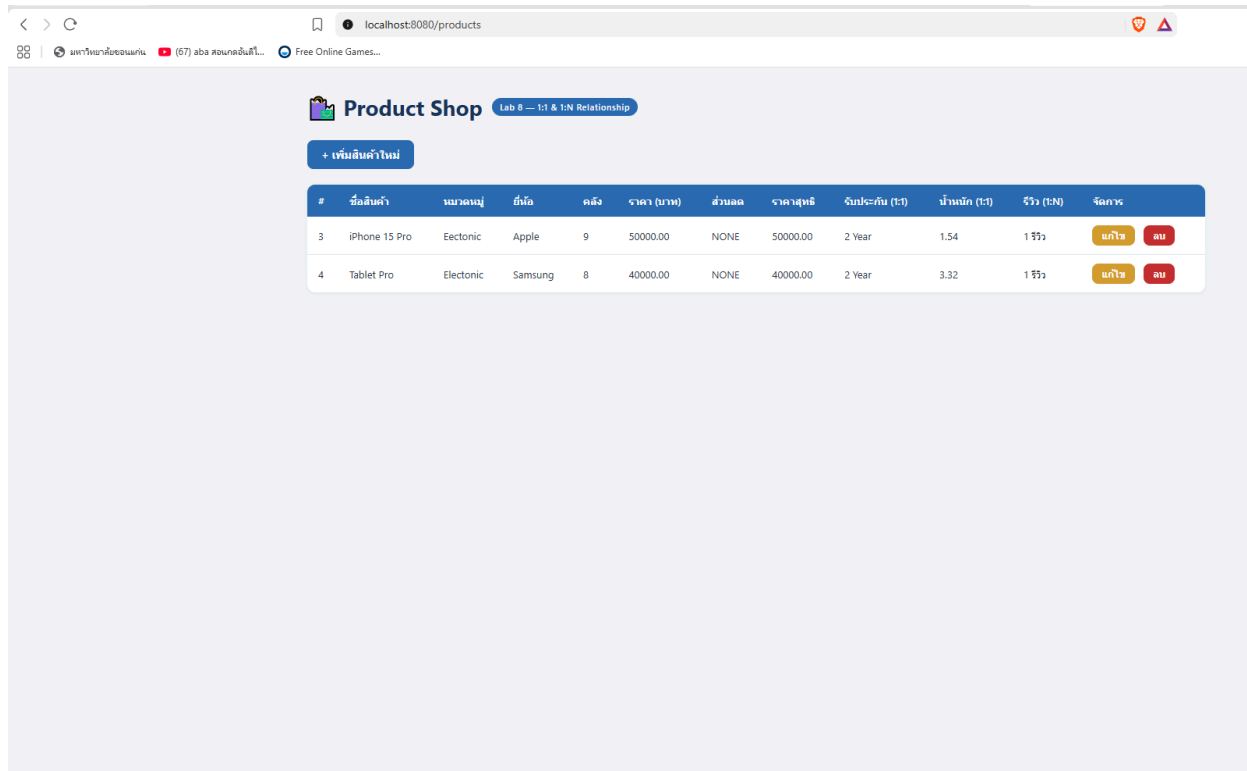
คะแนน (1-5)

ความเห็น

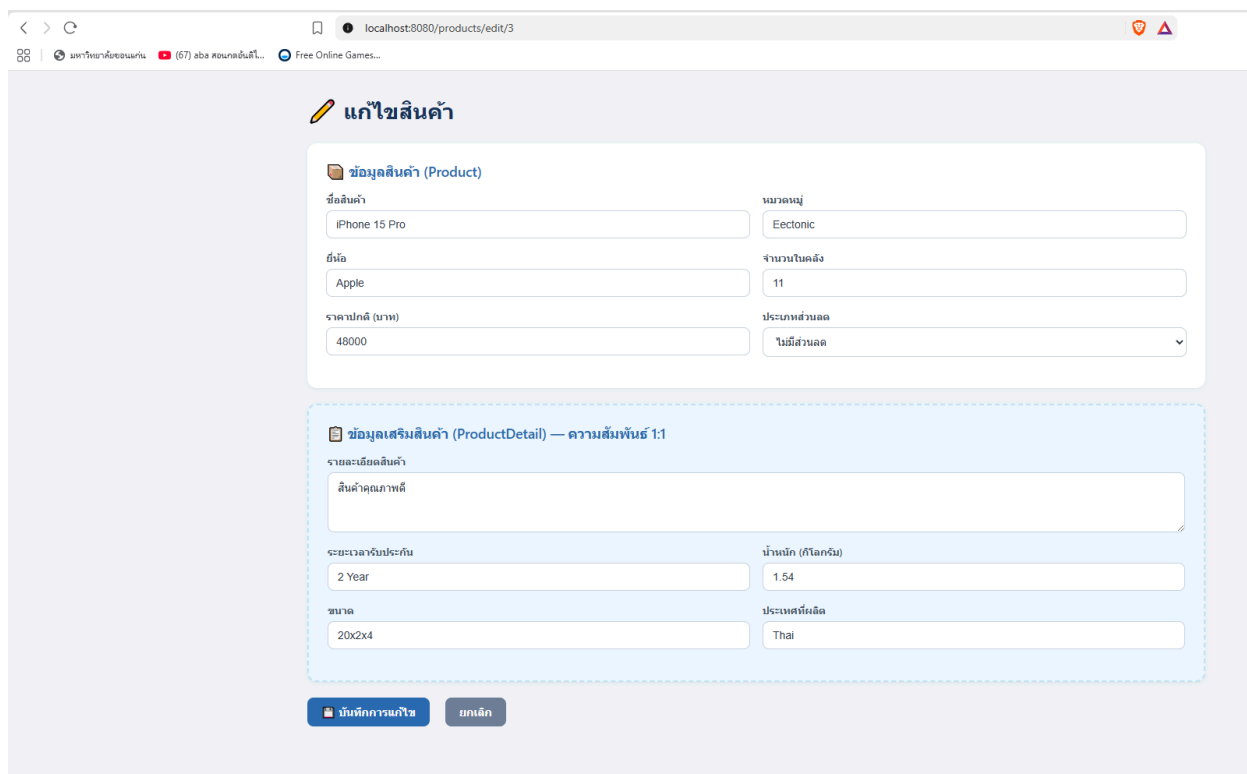
บันทึกสินค้า

ยกเลิก

หน้า list.html



หน้า edit.html



หน้า delete.html

localhost:8080/products/delete/4

หน้าร้านของฉัน (67) aba คอมพิวเตอร์... Free Online Games...

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้หรือไม่? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า: Tablet Pro

หมวดหมู่: Electronic

ยี่ห้อ: Samsung

ราคา: 40000.00 บาท

รายละเอียด: สินค้าใหม่

รับประกัน: 2 Year

ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกับตัว เนื่องจาก CascadeType.ALL

ยืนยันลบ

ยกเลิก

localhost:8080/products

หน้าร้านของฉัน (67) aba คอมพิวเตอร์... Free Online Games...

Product Shop

Lab 6 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	ประกัน (1:1)	รีวิว (1:N)	จัดการ
3	iPhone 15 Pro	Eectonic	Apple	11	48000.00	NONE	48000.00	2 Year	1.54	0 รีวิว	แก้ไขลบ

ตาราง products

pgAdmin 4

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (4)
 - lab7demo
 - lab7demo2
 - lab8shop
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (3)
 - product_details
 - products
 - reviews
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

Query

```
SELECT * FROM public.products
ORDER BY id ASC
```

Data Output

id	brand	category	discount_type	name	price	stock	detail_id
1	Apple	Electronic	NONE	iPhone 15 Pro	48000	11	11

Showing rows: 1 to 1 of 1

Page No: 1

CRLF Ln 1, Col 1

ตาราง product_details

pgAdmin 4

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (4)
 - lab7demo
 - lab7demo2
 - lab8shop
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (3)
 - product_details
 - products
 - reviews
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

Query

```
SELECT * FROM public.product_details
ORDER BY id ASC
```

Data Output

id	description	dimensions	manufactured_country	warranty	weight
1	สินค้าคอมพิวเตอร์	20x2x4	Thai	2 Year	1.54

Showing rows: 1 to 1 of 1

Page No: 1

CRLF Ln 1, Col 1

ตาราง reviews

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (4)
 - lab7demo
 - lab7demo2
 - lab8shop
 - Cast
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (3)
 - product_details
 - products
 - reviews
 - Trigger Functions
 - Types
 - Views
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

public.product_det... X public.products/la... X public.reviews/lab8shop/postgres@PostgreSQL 18 X

public.reviews/lab8shop/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM public.reviews
2 ORDER BY id ASC
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

id	comment	rating	review_date	reviewer	product_id
[PK] bigint	character varying (255)	integer	date	character varying (255)	bigint
1	6 673380067-2 sec.2	5	[null]	สัฟเวร์ ลักขณ	6

Total rows: 1 Query complete 00:00:00.088

Activate Windows
Go to Settings to activate Windows.

CRLF Ln 1, Col 1